

SUMMER COURSE

ROS2：架構、通訊模型與 TF

Week 4.5 · Middleware, Communication Patterns, and Coordinate Frames

獵奇緯



目錄

01 Part 0 · 似曾相似的架構設計

03 Part 2 · Topic

05 Part 4 · Action

02 Part 1 · DDS

04 Part 3 · Service

06 Part 5 · TF

SECTION 01

Part 0 · 似曾相似的架構設計



01

■ 一台機器人，其實是十幾支獨立的程式



觀察

一台真正在跑的機器人，底下是十幾支各自獨立的 process 同時運作：讀 LiDAR、讀相機、做定位、做規劃、控制馬達，甚至跑在不同電腦上。

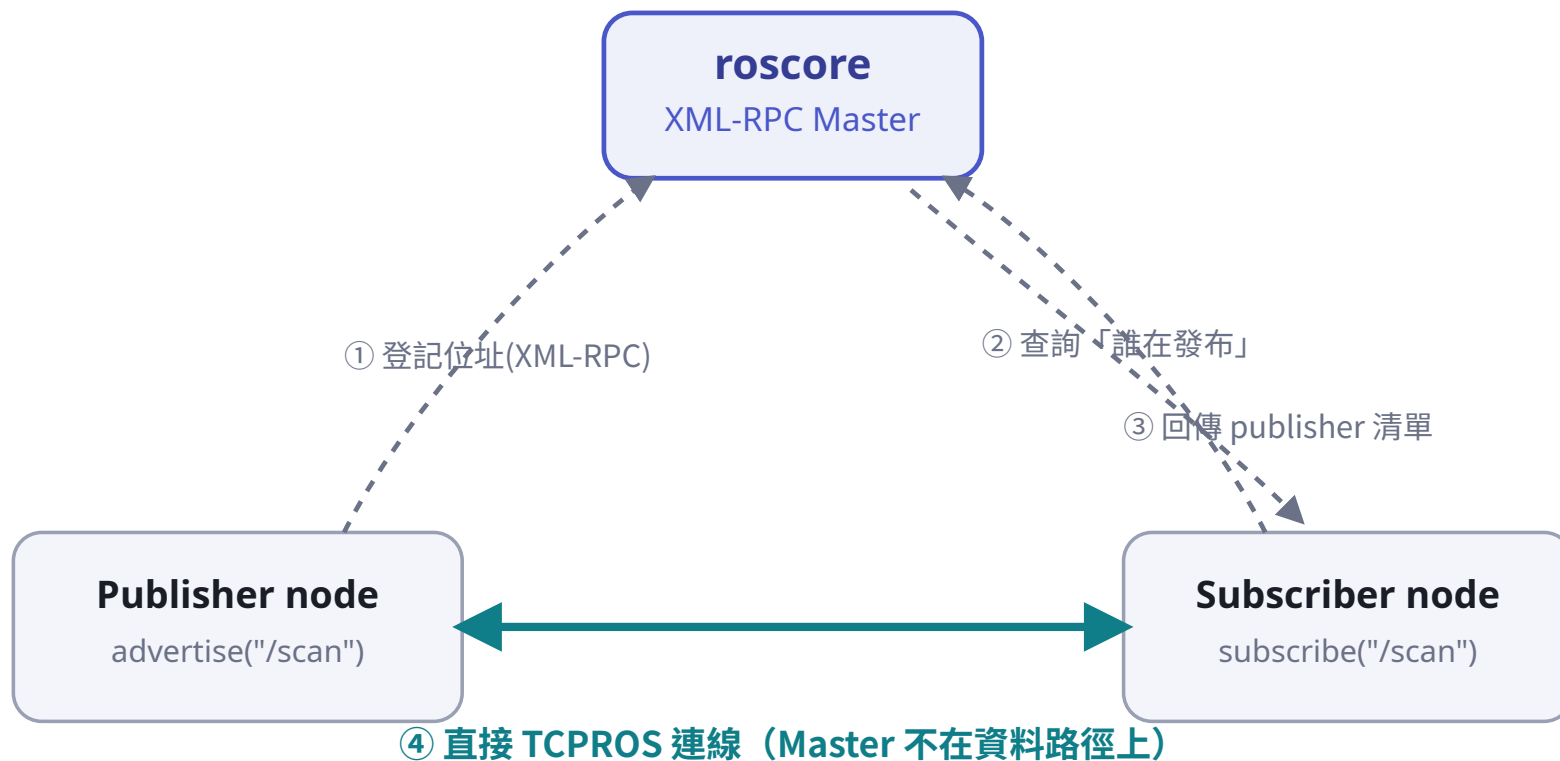
? 第一個問題

這些 process 要合作出一個連貫行為，第一件要解的事：一個 process 怎麼知道另一個 process 在哪裡、能提供什麼？

① Info

這個問題你並不陌生：作業系統課裡的主僕式（master-slave）架構，一台機器維護全域表，記錄誰擁有什麼資源，其他機器有事先去問它，你已經看過，只是場景通常是檔案系統或資源管理。

ROS 1：主僕式架構的具體實作



主僕架構的代價

① 單點協調者

`roscore` 掛掉，既有連線不斷，但任何「新加入」全部停擺：新開的 node 沒有地方登記、查詢。

② 每個節點都要知道位址

多機部署要對齊

`ROS_MASTER_URI` 、

`ROS_IP` / `ROS_HOSTNAME` ，

兜不起來是最常見的排錯來源之一。

③ 無法區分可靠性需求

TCPROS 底層只有一種行為，要嘛都保證送達、要嘛都不保證，沒有語彙可以逐一挑選。

✓ Tip

這三件事正是主僕式架構本身在分散式系統裡固有的代價。下一節換一種解法：把「誰知道全貌」從一個特定節點，換成讓每個節點自己去認識彼此。

SECTION 02

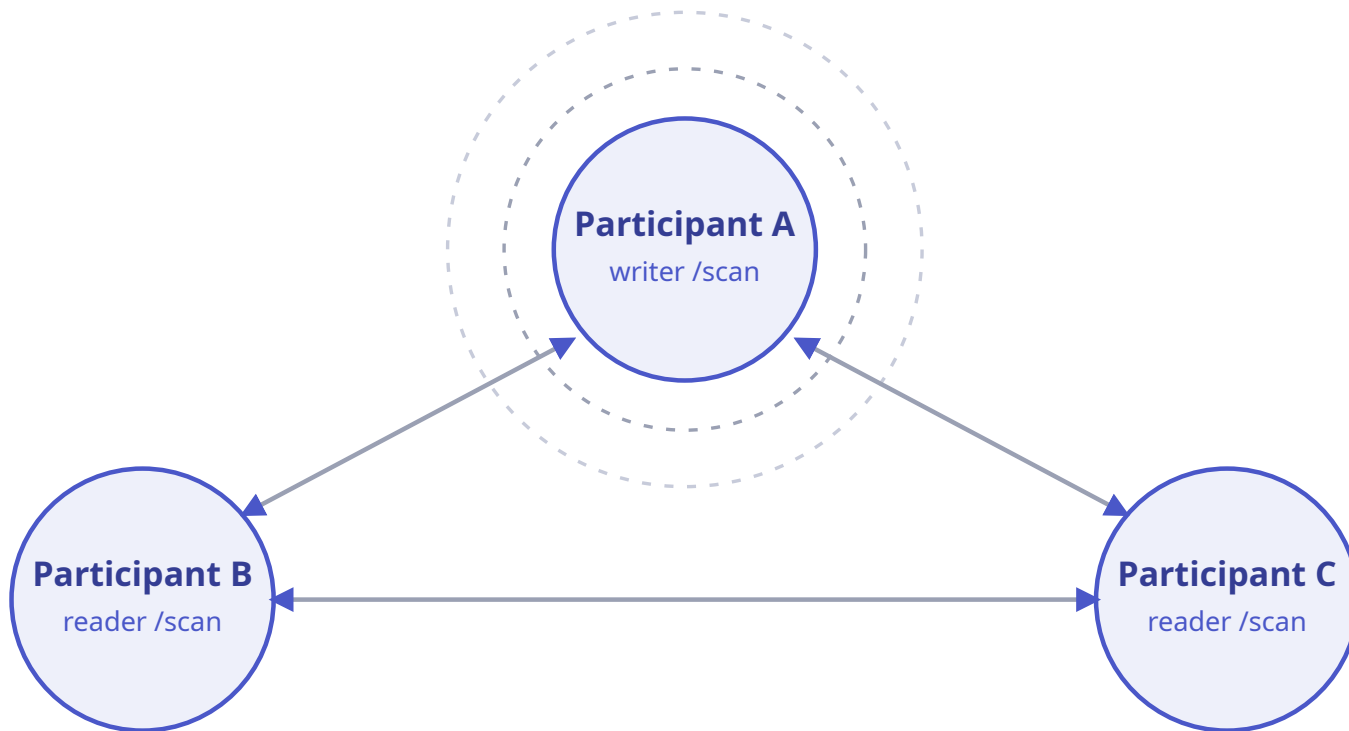
Part 1 · DDS



02

去中心化的發現協定

週期性 multicast 廣播自我描述 (PDP) ，沒有中央仲裁者



✓ A(RELIABLE) ↔ B(RELIABLE) 相容

✗ A(RELIABLE) ↔ C(BEST_EFFORT) 不相容

QoS 相容是一個偏序關係

$\text{match}(r, w)$ iff $\text{name}(r) = \text{name}(w)$, $\text{type}(r) = \text{type}(w)$, $\text{QoS}(r) \sqsubseteq \text{QoS}(w)$

✓ 相容

reader 要求 `RELIABLE` , writer 承諾 `RELIABLE` (保證送達、必要時重傳) 。
writer 給得出 reader 要求的保證。

✗ 不相容

reader 要求 `RELIABLE` , writer 只承諾 `BEST_EFFORT` (送了就算, 不重傳) 。
reader 要求的保證, writer 給不出來。

⚠ 注意

現場排錯症狀：兩個 node 在 `ros2 node info` 裡互相看到對方，拓樸正常，卻死活收不到訊息，幾乎都是 QoS 不相容。「拓樸正常但沒資料」 $\approx$ 「去檢查 QoS」。

| rmw 抽象層

分層

`rclcpp` / `rclpy` → `rcl` → `rmw` → 特定 DDS 實作（Fast-DDS、Cyclone DDS、RTI Connnext …）。上層寫的只是介面，跟 OS 課裡系統呼叫抽象掉裝置驅動是同一種手法。

① Info

抽象不是免費的：multicast 是否可用、discovery storm 的頻寬開銷，最終仍取決於底下真正跑的是哪一個 DDS 實作。節點數超過幾十個時，discovery 流量本身會變成要調校的效能問題。

SECTION 03

Part 2 · Topic



03

Topic 很單薄，複雜度全下推到 DDS

KEEP_LAST(n) (預設)

只保留最新的 `n` 筆，佇列滿了就丟最舊的一筆。`n` 預設 10。

KEEP_ALL

試圖保留所有還沒被消費的訊息，直到觸及 resource limit。

① Info

Depth 決定的是：subscriber 處理速度跟不上 publisher 時，哪一筆訊息會被留下來。正常運作時「最新一筆」跟「照時間順序不漏接」看起來一樣，系統過載時兩者的行為就會分岔。

Executor：誰、什麼時候真的處理這筆訊息

SingleThreadedExecutor：一個 process，一條時間軸

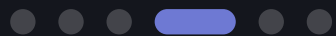


佇列裡的 callback 依序輪流執行，同一時間只跑一個。

前一個沒返回，後面全部插不了隊，不論它們原本多重要。

SECTION 04

Part 3 · Service



04

■ 建立在 pub/sub 之上的請求回應

→ Topic

單向資料流，誰發布、誰訂閱不需要對應關係。

⇔ Service

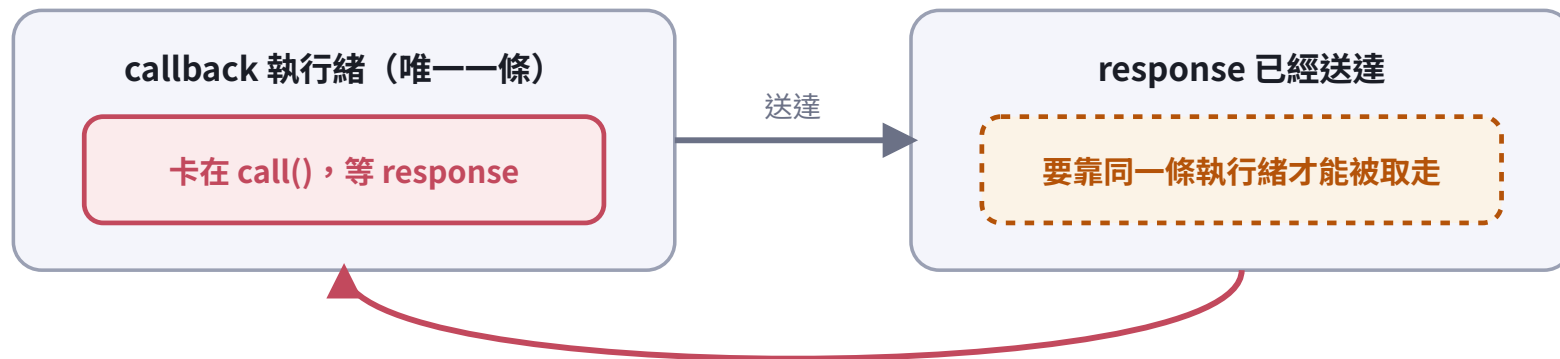
一來一往的請求回應，一個 request 對應恰好一個 response。

① Info

實作上，service 底層仍然建立在 DDS 的 pub/sub 之上：每個 service 背後有一對 DDS topic (request / response)，透過 sequence number 跟 client GUID 互相關聯。ROS2 沒有另造 RPC 機制，是用同一套 pub/sub 語彙疊上關聯資訊，模擬出請求回應的語意。

同步呼叫在單執行緒 executor 裡的死結

單執行緒 executor 裡的同步 service 呼叫



但要處理它，得先讓 executor 迴圈繼續跑，而執行緒正被 call() 佔用

死結：兩邊都在等對方先讓步

SECTION 05

Part 4 · Action



05

三個 service 加兩個 topic

① goal service

client 發起，server 決定接受或拒絕。

② cancel service

client 中途放棄，server 決定能否安全中止。

③ get_result service

任務結束後，取得最終結果。

feedback topic

持續發布進度資訊，純粹 pub/sub。

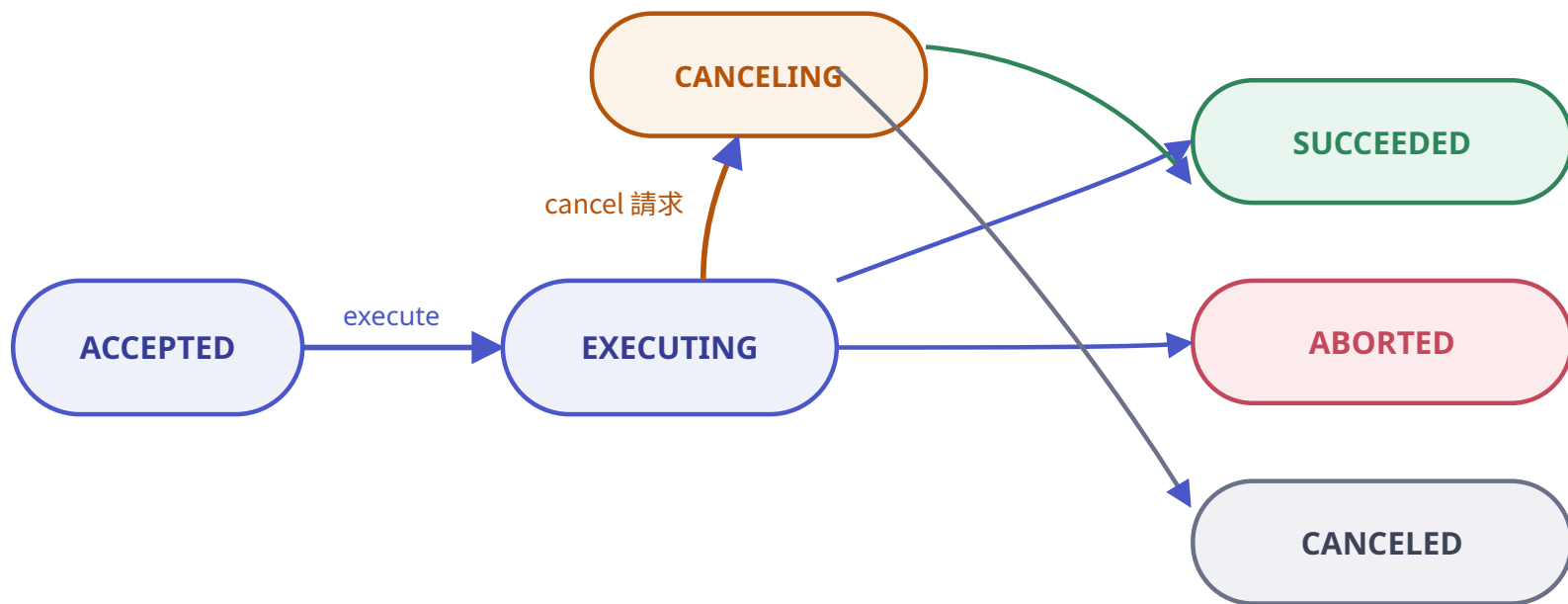
status topic

回報目前狀態機所在的狀態。

✓ Tip

Action 沒有引入任何新的底層通訊機制，理解這個組合關係，比死記 API 更重要：往後 action 卡住，都可以拆回「這其實是哪一個 service 或 topic 出問題」來排查。

Action 的有限狀態機



終止狀態一旦進入就不可逆，get_result service 回傳其中一個。

SECTION 06

Part 5 · TF



06

TF 難在兩種思考方式要同時成立

Σ 數學

把每一次量測、每一個感測器安裝位置，精確地寫成 $SE(3)$ 裡的一個元素，照 Week 4 的規則做合成、取逆。

⚙ 系統實作

這些 $SE(3)$ 元素由不同 node、以不同頻率、在不同時刻發布出來，發布本身要遵守拓樸規則。

⚠ 注意

一行程式碼出錯，要分辨到底是數學錯了（合成順序、取逆、旋轉慣例弄反），還是時序錯了（query 到的那筆資料根本不是你以為的那個時刻）？

座標系的樹，邊是 SE(3) 元素

$$T^{-1} = (R^T, -R^T p)$$

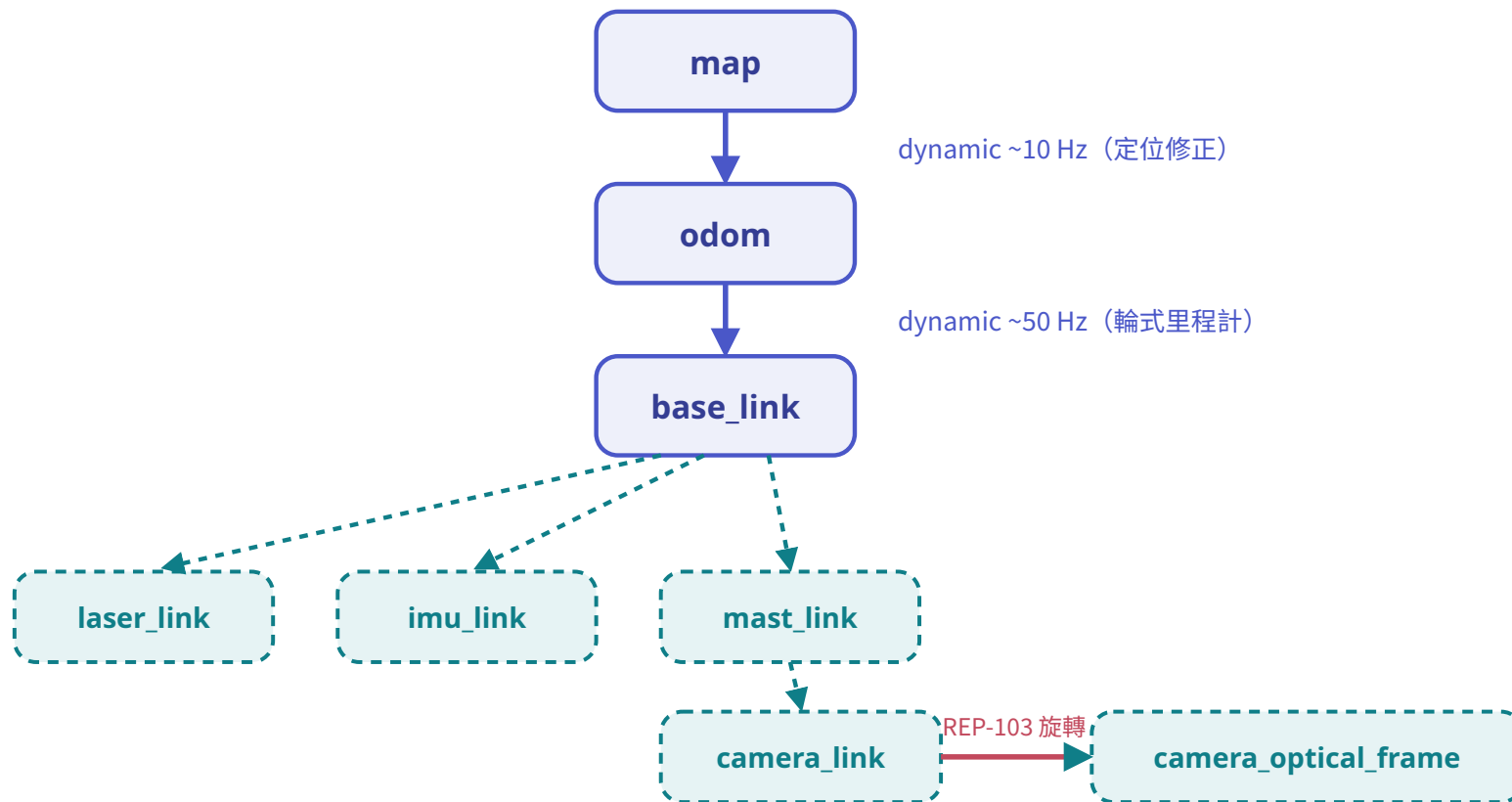
Info

TF 把機器人跟它的感測器表示成一個有向樹：每一個節點是一個座標系（frame），每一條邊是一個 rigid transform，精確來說是 Week 4 定義過的 SE(3) 元素，由旋轉 R 跟平移 p 構成。

Tip

要問「A 相對於 B 的 pose 是什麼」：沿著樹走到兩者的最近共同祖先，把沿途每一條邊的 SE(3) 元素依序合成。方向跟父子關係相反的邊，合成前先取逆，用的正是上面這條 Week 4 已經證過的公式。

AMR 的座標樹



實線 = dynamic edge (隨時間變化，走 /tf) 虛線 = static edge (發布一次，走 /tf_static，TRANSIENT_LOCAL)

Static 與 Dynamic : QoS 又出場一次

	Static (laser_link 、 imu_link ...)	Dynamic (odom → base_link)
Topic	/tf_static	/tf
廣播頻率	一次	持續，每筆帶時間戳
QoS durability	TRANSIENT_LOCAL	VOLATILE (預設)
遲加入的訂閱者	直接拿到最後一筆	要等下一次廣播

時間戳把 lookup 變成一個插值問題

$$\alpha = \frac{t - t_0}{t_1 - t_0}, \quad p(t) = \text{lerp}(p_0, p_1; \alpha), \quad R(t) = \text{slerp}(R_0, R_1; \alpha)$$

Info

`slerp` 精確地是 Week 4 提過的、在 `SO(3)` 上以固定角速度沿測地線走的插值。

Tip

若 `t` 落在時間窗外，Buffer 拒絕外插，丟出例外，把「這筆資料到底存不存在」明確地交還給呼叫者，不會偷偷回傳一個瞎猜的值。

TimePointZero 的陷阱

✓ 正確配對

掃描匹配的答案描述 `t-150ms` 那一刻，就去 Buffer 查 `t-150ms` 時刻的里程計樣本，兩者在同一時刻為真。

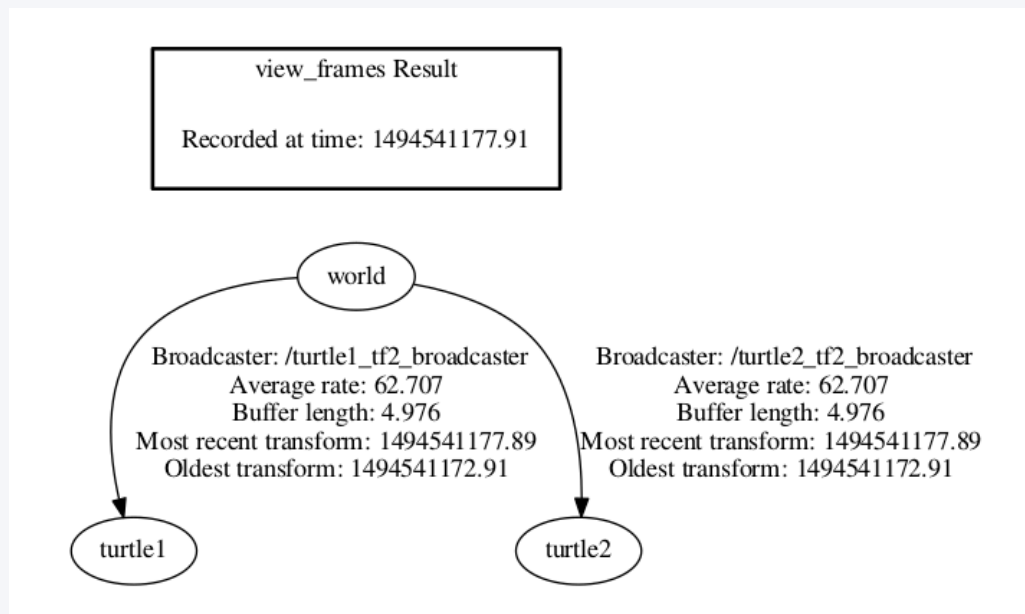
✗ TimePointZero 陷阱

傳入 `TimePointZero`：「不管哪個時刻，給我最新一筆」，直接跳過插值，拿到的是查詢當下的最新樣本。

✗ 危險

把「最新一筆」拿去跟另一個帶明確時間戳的量測合成，兩個 `SE(3)` 元素從來沒有在同一時刻為真過。合成結果型別上完全合法，數值上卻是兩個不同時刻拼出來的假象，跟 Week 4 複數平均 179° 與 -179° 的陷阱，是同一種錯誤的不同臉孔。

誰能發布哪條邊：map $\rightarrow$ odom 的修正模式



真實的 `view_frames` 輸出（turtlesim 範例），可看到每條邊各自的發布頻率與緩衝長度

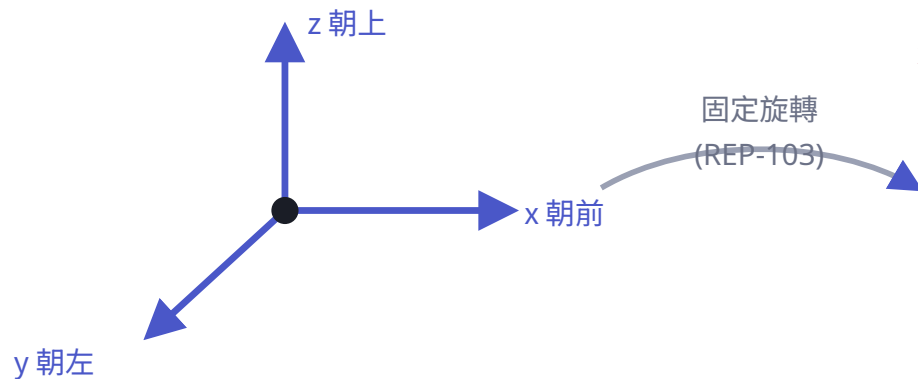
Info

同一條邊、同一時刻，只能有一個發布者。輪式里程計獨佔

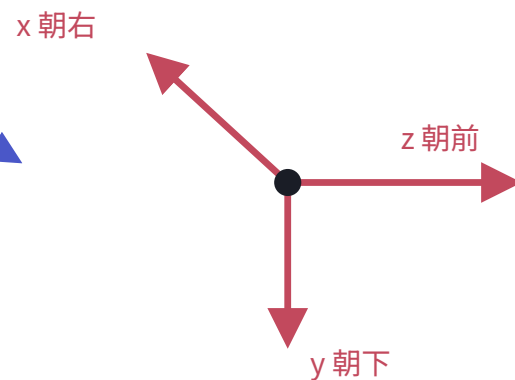
`odom  $\rightarrow$  base_link`（毫秒級頻率），
定位獨佔 `map  $\rightarrow$  odom`（慢、有處理延遲的修正量）。兩個節點各自只負責自己那條邊。

旋轉慣例的陷阱：REP-103

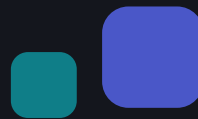
camera_link (機械慣例)



camera_optical_frame (視覺慣例)



漏掉這個固定旋轉，樹照樣連通、lookup 照樣有回傳值，只是方向是錯的。



Q & A

獵奇緯 · Summer Course